

GOOGLE DORK / OSINT SECURITY ASSESSMENT REPORT

Assessment Type: Passive Google Dork / OSINT Reconnaissance

Prepared By: Amaresh Javali

Assessment Period: [10-09-2026]

Scope: Publicly indexed assets identified via passive search-engine reconnaissance

1. Executive Summary

This report presents the results of a passive, search-engine-driven (Google Dork / OSINT) reconnaissance assessment. The objective was to identify information about the target organizations and related assets that is already publicly indexed by search engines, and to evaluate which of those observations represent genuine, actionable security vulnerabilities as opposed to informational exposure of content that is intended to be public or that requires further verification.

During this assessment, 10 observations were identified. Of these, 5 represent confirmed technical security vulnerabilities that warrant remediation, and 5 represent informational or reconnaissance-value observations — several of which describe content that appears to be intentionally public or whose sensitivity could not be confirmed without further authorization. Section 4 lists all observations together with their severity; Sections 5 and 6 present the confirmed vulnerabilities and informational observations separately, using a standardized reporting template for every entry.

SEVERITY BREAKDOWN

Critical	High	Low	Informational
2	3	2	3

The two highest-severity items — an unauthenticated live camera feed (F-03) and a publicly exposed .env file containing database credentials (F-04) — should be treated as immediate priorities. Full detail, evidence, and remediation guidance for every finding is provided in Sections 5 and 6.

2. Assessment Scope and Authorization

This assessment was limited to passive reconnaissance using publicly available search-engine results (Google Dorking / OSINT techniques). No authentication attempts, vulnerability exploitation, credential use, brute-force activity, unauthorized downloads, content modification, or denial-of-service testing were performed against any target. All findings in this report reflect information that was already indexed and publicly reachable at the time of testing; no access controls were bypassed to obtain them.

Where a finding references an organization or domain, this report does not assert that the assessment was formally authorized by that organization; targets were selected as illustrative, real-world examples of exposure discoverable through passive search-engine reconnaissance. Any recipient acting on these findings should confirm their own authorization before taking remediation action on systems they do not own or operate.

3. Assessment Methodology

All findings in this report were identified using passive reconnaissance only — no direct interaction with target systems beyond standard, unauthenticated HTTP requests made by a web browser to view already-indexed pages. The following approach was used throughout:

- Search-engine queries (Google Dorks) were used to filter publicly indexed content down to categories of interest, such as exposed directories, configuration files, and login portals.
- Every result was manually reviewed in a browser to confirm that the content was genuinely reachable and to capture evidence.
- No credentials were entered, guessed, or reused against any system.
- No files were downloaded and analyzed beyond what was necessary to view content already rendered by the browser.
- No scanning, brute-forcing, exploitation, or denial-of-service activity was performed at any point.

Testing limitations: because only passive techniques were used, this assessment does not confirm whether any exposed credential is still valid, whether any exposed directory contains additional non-public files beyond what was visible, or whether any identified issue has since been remediated. Each finding's Impact Assessment section separates what was directly Observed, what was Demonstrated during testing, and what remains a Potential (unverified) impact, so that severity ratings are not inflated beyond what the evidence supports.

4. Findings Summary

The table below lists every observation from this assessment. Findings F-01 through F-05 are confirmed technical vulnerabilities; findings F-06 through F-10 are informational or reconnaissance-value observations, several of which appear to describe intentionally public content. Full detail for each is provided in Sections 5 and 6.

ID	Finding	Category	Severity
F-01	Exposed Backend Source Code Directory	Confirmed Vulnerability	High
F-02	Publicly Accessible Git Repository (.git)	Confirmed Vulnerability	High
F-03	Unauthenticated Live Camera Feed	Confirmed Vulnerability	Critical
F-04	Exposed .env File Containing Database Credentials	Confirmed Vulnerability	Critical
F-05	Potentially Sensitive Password-Related Files in a Public Directory	Confirmed Vulnerability (Potential)	High (Potential)
F-06	Publicly Indexed Administrative Login Page	Informational Observation	Informational
F-07	Information Disclosure via robots.txt	Informational Observation	Low
F-08	TxDOT Public Plans Directory Listing	Informational Observation (Likely Intentional)	Informational
F-09	Publicly Indexed Subdomains and Login Portals	Informational Observation	Low
F-10	Publicly Accessible Corporate Financial Document	Informational Observation	Informational

F-01 — Exposed Backend Source Code Directory

Category	Confirmed Vulnerability
Severity	High
CWE	CWE-538 (File and Directory Information Exposure), CWE-540 (Inclusion of Sensitive Information in Source Code)
OWASP Category	A05:2021 - Security Misconfiguration
Target	tmrees.org
Path / URL	/concrete/src/User/
Discovery Method	Passive reconnaissance (search-engine indexing)
Dork Used	intitle:"index of" /concrete/password

DESCRIPTION

A Google Dork search revealed a directory on the target website with directory listing enabled. The exposed directory contains backend PHP source code for the site's login and user-management system, which is built on Concrete CMS. Visible files include logic for login handling, password validation, registration, and session/authentication management.

EVIDENCE

Name	Last modified	Size	Description
Parent Directory		-	
Avatar/	2026-02-25 15:52	-	
CsvWriter.php	2021-06-17 21:14	3.5K	
EditResponse.php	2021-06-17 21:14	716	
Event/	2026-02-25 15:52	-	
Exception/	2026-02-25 15:52	-	
Group/	2026-02-25 15:52	-	
LogSubscriber.php	2021-06-17 21:14	2.7K	
Login/	2026-02-25 15:52	-	
Notification/	2026-02-25 15:52	-	
Password/	2026-02-25 15:52	-	
PersistentAuthentica...>	2026-02-25 15:52	-	
Point/	2026-02-25 15:52	-	
PostLoginLocation.php	2021-06-17 21:14	7.3K	
PrivateMessage/	2026-02-25 15:52	-	
RegistrationService.php	2021-06-17 21:14	7.0K	
RegistrationServiceI...>	2021-06-17 21:14	237	
Search/	2026-02-25 15:52	-	
Statistics.php	2021-06-17 21:14	760	
StatusService.php	2021-06-17 21:14	1.6K	
StatusServiceInterfa...>	2021-06-17 21:14	122	
User.php	2021-06-17 21:14	34K	
UserBannedIp.php	2021-06-17 21:14	1.8K	
UserInfo.php	2021-06-17 21:14	37K	
UserInfoRepository.php	2021-06-17 21:14	3.7K	
UserList.php	2021-06-17 21:14	15K	
UserServiceProvider.php	2021-06-17 21:14	3.8K	
ValidationHash.php	2021-06-17 21:14	3.1K	
Workflow/	2026-02-25 15:52	-	

Figure 1 – Directory listing of /concrete/src/User/ showing publicly browsable PHP source files.

IMPACT ASSESSMENT

Observed: The directory is publicly browsable and lists PHP source files belonging to the site's authentication and user-management components, along with the CMS platform in use (Concrete CMS).

Demonstrated: No source files were downloaded, opened, or analyzed. The finding is limited to confirming that the directory listing is publicly reachable and that the file names indicate authentication-related code.

Potential: If the listed files were reviewed, they could reveal how login and session logic is implemented, the exact CMS version in use (allowing lookup of known, version-specific exploits), and potentially hardcoded credentials or API keys left in the code. This would materially lower the effort required for an attacker to plan further attacks against the login system.

RECOMMENDATION

- Disable directory listing on the web server.
- Block direct external access to .php source files and other backend directories.
- Update the CMS to the latest supported version.
- Review the exposed code for hardcoded credentials or API keys and rotate any that are found.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. The directory listing was viewed as presented by the browser. No files were downloaded, executed, or analyzed, and no exploitation was attempted.

[Responsible Disclosure Message :](#)

Subject: **Exposed Source Code Directory on tmrees.org**

Hello Security Team,

During passive Google-dork reconnaissance, I found that a directory at `/concrete/src/User/` on `tmrees.org` is publicly browsable and exposes Concrete CMS backend PHP source code, including login and authentication logic.

No attempt was made to access, modify, or exploit the exposed information beyond confirming the finding.

I recommend disabling directory listing on the web server, blocking direct access to `.php` source files, and updating the CMS to its latest version.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

F-02 — Publicly Accessible Git Repository (.git)

Category	Confirmed Vulnerability
Severity	High
CWE	CWE-527 (Exposure of Version-Control Repository to an Unauthorized Control Sphere), CWE-538
OWASP Category	A05:2021 - Security Misconfiguration
Target	kmoodle.su.edu.eg
Path / URL	/.git/
Discovery Method	Passive reconnaissance (search-engine indexing)
Dork Used	intitle:"index of" inurl:.git

DESCRIPTION

The application's .git directory is publicly accessible without authentication, and the server has directory listing enabled. This exposes Git repository internals, including HEAD, config, index, objects/, logs/, and refs/. A .git directory should exist only in a developer's local working copy or a private repository host; it should never be reachable from a production web server, because it contains the complete version-control metadata for the project.

EVIDENCE

Index of /.git

Name	Last modified	Size	Description
 Parent Directory		-	
 HEAD	2021-04-05 14:58	34	
 branches/	2021-04-05 14:58	-	
 config	2021-04-05 14:58	340	
 description	2021-04-05 14:58	73	
 hooks/	2021-04-05 14:58	-	
 index	2021-04-05 14:58	2.5M	
 info/	2021-04-05 14:58	-	
 logs/	2021-04-05 14:58	-	
 objects/	2021-04-05 14:58	-	
 packed-refs	2021-04-05 14:58	41K	
 refs/	2021-04-05 14:58	-	

Apache/2.4.29 (Ubuntu) Server at kmoodle.su.edu.eg Port 443

Figure 2 – Directory listing of the publicly reachable .git folder, showing Git internal metadata.

IMPACT ASSESSMENT

Observed: The .git directory and its internal folder structure (HEAD, config, objects, logs, refs) are publicly listed and browsable without authentication.

Demonstrated: The directory listing was viewed as presented by the browser. The repository was not downloaded, cloned, or reconstructed, and its contents were not analyzed for secrets.

Potential: A fully exposed .git directory can typically be downloaded in its entirety using publicly available tools, which can hand an attacker a complete copy of the application's source history. This could include prior versions of files removed for security reasons, internal endpoints and configuration, commit authors and messages, and accidentally committed credentials or API keys. The actual impact depends on the specific contents of the repository, which were not examined during this assessment.

RECOMMENDATION

- Remove the .git directory from the production web root entirely.
- Explicitly deny access to /.git/ at the web-server level (e.g., Apache/Nginx location rules).
- Disable unnecessary directory indexing on the web server.
- Review the full Git history for accidentally committed sensitive information.
- Rotate any credentials, keys, or secrets found in the repository history.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. The directory listing was viewed as presented by the browser. The repository was not downloaded, cloned, or analyzed for secrets, and no exploitation was performed.

Responsible Disclosure Message :

Subject: **Publicly Accessible .git Directory**

Hello Security Team,

During passive reconnaissance, I found that the .git directory at <https://kmoodle.su.edu.eg/.git/> is publicly browsable with directory listing enabled, exposing Git metadata such as HEAD, config, objects, logs, and refs.

No attempt was made to access, modify, or exploit the exposed information beyond confirming the finding.

I recommend removing the .git directory from the production web root, blocking access to it at the web-server level, and rotating any credentials found in the repository history.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

F-03 — Unauthenticated Live Camera Feed

Category	Confirmed Vulnerability
Severity	Critical
CWE	CWE-306 (Missing Authentication for Critical Function)
OWASP Category	A01:2021 - Broken Access Control
Target	Undisclosed host (default IP camera/DVR web interface)
Path / URL	/view/view.shtml
Discovery Method	Passive reconnaissance (search-engine / IoT search indexing)
Dork Used	inurl:/view/view.shtml

DESCRIPTION

A live camera interface is publicly reachable without any authentication. The endpoint pattern /view/view.shtml is a well-known default web interface path used by a range of IP camera and DVR/NVR products, and is commonly indexed by search engines and IoT-focused search tools when devices are deployed without any access restriction.

EVIDENCE

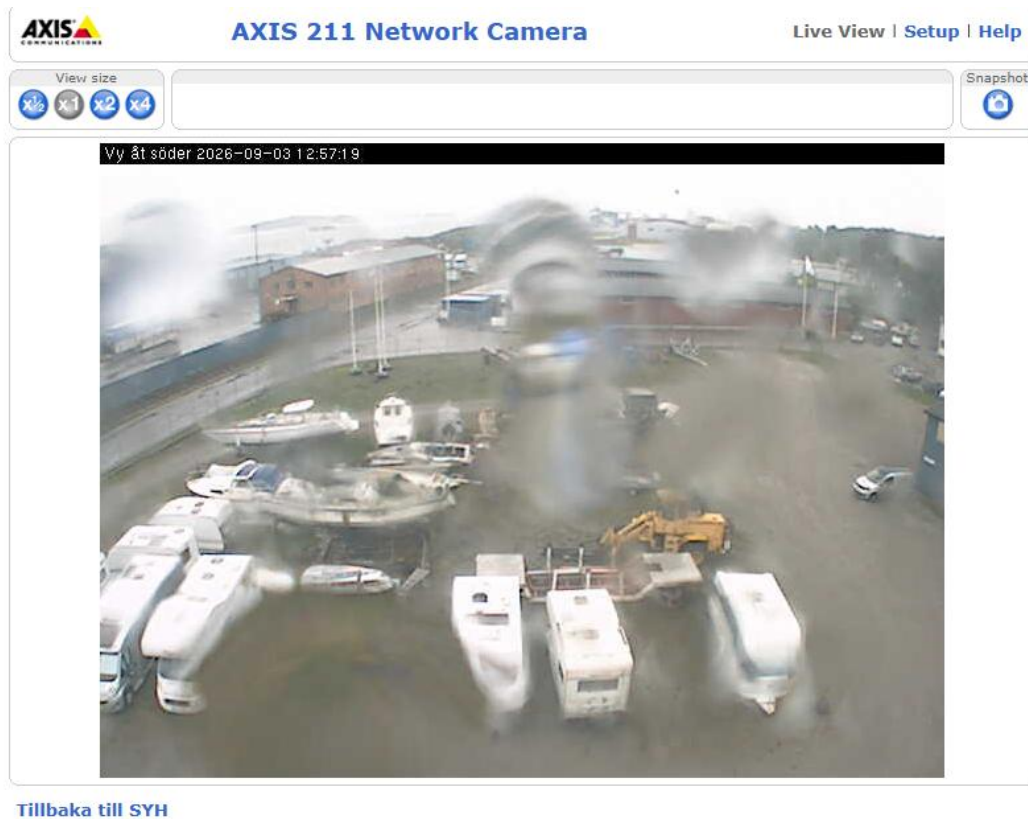


Figure 3 – Camera web interface at /view/view.shtml, showing a live feed with no authentication prompt.

IMPACT ASSESSMENT

Observed: Navigating directly to the endpoint displays the camera's live video feed with no login prompt or access control of any kind.

Demonstrated: Access to the live feed was confirmed by viewing it in a browser. No configuration changes, pan-tilt-zoom (PTZ) commands, snapshots, or administrative functions were attempted.

Potential: Anyone who discovers the endpoint can view live video without permission, monitor people, locations, or activity, and gather information about the physical environment. If the camera exposes additional unauthenticated functionality (PTZ control, snapshot capture, audio, or administrative settings), the impact could extend to active manipulation of the device or a pivot toward broader network compromise; this was not tested.

RECOMMENDATION

- Require authentication before the camera feed can be accessed.
- Restrict the camera interface to authorized networks (internal network, VPN, or IP allow-listing).
- Disable public internet exposure where it is not operationally required.
- Review access controls and disable unnecessary remote-management functionality.
- Update camera firmware/software and review access logs for unauthorized access.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. The feed was viewed in a browser to confirm the absence of authentication. No PTZ, snapshot, audio, or administrative functions were tested, and no configuration was changed.

Responsible Disclosure Message :

Subject: **Unauthenticated Live Camera Feed Exposure**

Hello Security Team,

During passive reconnaissance, I found that a live camera interface at `/view/view.shtml` is publicly reachable without any authentication, allowing the live video feed to be viewed by anyone.

No attempt was made to access, modify, or exploit the exposed information beyond confirming the finding.

I recommend requiring authentication before the camera feed can be accessed and restricting the interface to authorized networks only.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

F-04 — Exposed .env File Containing Database Credentials

Category	Confirmed Vulnerability
Severity	Critical
CWE	CWE-798 (Use of Hard-coded Credentials), CWE-312 (Cleartext Storage of Sensitive Information)
OWASP Category	A02:2021 - Cryptographic Failures
Target	huggingface.co/ArcRay/dify_docker_arm
Path / URL	/blob/main/.env
Discovery Method	Passive reconnaissance (search-engine indexing)
Dork Used	filetype:.env "password"

DESCRIPTION

A publicly accessible .env configuration file was discovered containing database connection credentials. The file exposes a PostgreSQL username, password, hostname, port, and database name, and appears to belong to a Dify deployment. This is a sensitive-information-disclosure / hardcoded-credentials issue.

EVIDENCE

```

173 # API Tool configuration
174 API_TOOL_DEFAULT_CONNECT_TIMEOUT=10
175 API_TOOL_DEFAULT_READ_TIMEOUT=60
176
177
178 # -----
179 # Database Configuration
180 # The database uses PostgreSQL. Please use the public schema.
181 # It is consistent with the configuration in the 'db' service below.
182 # -----
183
184 DB_USERNAME=postgres
185 DB_PASSWORD=difyai123456
186 DB_HOST=db
187 DB_PORT=5432
188 DB_DATABASE=dify
189 # The size of the database connection pool.
190 # The default is 30 connections, which can be appropriately increased.
191 SQLALCHEMY_POOL_SIZE=30
192 # Database connection pool recycling time, the default is 3600 seconds.
193 SQLALCHEMY_POOL_RECYCLE=3600
194 # Whether to print SQL, default is false.
195 SQLALCHEMY_ECHO=false
196
197 # Maximum number of connections to the database
198 # Default is 100
199 #
200 # Reference: https://www.postgresql.org/docs/current/runtime-config-connection.html#GUC-MAX-CONNECTIONS
201 POSTGRES_MAX_CONNECTIONS=100
202
203 # Sets the amount of shared memory used for postgres's shared buffers.
204 # Default is 128MB
205 # Recommended value: 25% of available memory
206 # Reference: https://www.postgresql.org/docs/current/runtime-config-resource.html#GUC-SHARED-BUFFERS
207 POSTGRES_SHARED_BUFFERS=128MB
208
209 # Sets the amount of memory used by each database worker for working space.
210 # Default is 4MB
211 #
212 # Reference: https://www.postgresql.org/docs/current/runtime-config-resource.html#GUC-WORK-MEM
213 POSTGRES_WORK_MEM=4MB
214

```

Figure 4 – Publicly accessible .env file showing PostgreSQL connection credentials.

IMPACT ASSESSMENT

Observed: The .env file is publicly readable and contains plaintext PostgreSQL connection parameters, including username and password.

Demonstrated: The credentials were observed in the file. No connection attempt was made to the associated database or any other service, so the current validity of the credentials was not confirmed.

Potential: If the credentials are still valid and the associated database is reachable, an unauthorized party could gain access to the database. If the password has been reused elsewhere, additional systems could also be at risk.

RECOMMENDATION

- Immediately rotate/revoke the exposed password.
- Remove the secret from the public repository, including from Git history if it was previously committed.
- Add .env to .gitignore going forward.
- Store secrets in environment variables managed by the deployment platform or a dedicated secrets manager, not in a committed file.

- Review whether the exposed credentials were reused on other systems.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. The credentials were viewed but not used to authenticate to any system, database, or service.

Responsible Disclosure Message :

Subject: **Exposed .env File Containing Database Credentials**

Hello Security Team,

During passive reconnaissance, I found a publicly accessible .env file in the linked Hugging Face repository containing PostgreSQL database credentials, including username, password, host, port, and database name.

No attempt was made to access, modify, or exploit the exposed information beyond confirming the finding.

I recommend immediately rotating the exposed credentials, removing the file from the repository and its Git history, and adding .env to .gitignore going forward.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

F-05 — Potentially Sensitive Password-Related Files in a Public Directory

Category	Confirmed Vulnerability (Potential)
Severity	High (Potential)
CWE	CWE-538 (File and Directory Information Exposure)
OWASP Category	A05:2021 - Security Misconfiguration
Target	wikileaks.org (hosted archive)
Path / URL	/sony/docs/bonus/1/Password/
Discovery Method	Passive reconnaissance (search-engine indexing)
Dork Used	filetype:txt "password"

DESCRIPTION

A Google Dork search using filetype:txt "password" surfaced a publicly accessible directory containing numerous files with names suggestive of stored authentication information, including Accounts Passwords.txt, Accounts and Passwords.xlsx, All NT_passwords.txt, Computer Passwords.txt, Database Passwords.txt, Digital User Name and passwords.txt, FTP LOGIN PASSWORD SHEET.txt, Password History_09132007.txt, and Change_password to access TWS server.txt, among other .xlsx, .pdf, .doc, and .txt files.

EVIDENCE

Index of /bonus/1/Password/

120302 MASTERS - DAX User List NO PASSWORDS.txt	01-Jan-1970 00:01	3335
120302 MASTERS - DAX User List NO PASSWORDS.xls...>	01-Jan-1970 00:01	51636
17 - Password History_09132007.pdf	01-Jan-1970 00:01	32551
17 - Password History_09132007.txt	01-Jan-1970 00:01	340
3N Notification password_userid.doc.pdf	01-Jan-1970 00:01	14311
3N Notification password_userid.txt	01-Jan-1970 00:01	1518
50 new user password.txt	01-Jan-1970 00:01	4009
50 new user password.xls.pdf	01-Jan-1970 00:01	33911
6 PWC password test.doc.pdf	01-Jan-1970 00:01	50207
6 PWC password test.txt	01-Jan-1970 00:01	125
90 Day Admin password.xlsx	01-Jan-1970 00:01	8007
ALL_SSL_Certs_2012.xlsx	01-Jan-1970 00:01	58183
Accounts Passwords.txt	01-Jan-1970 00:01	376
Accounts and Passwords.xlsx	01-Jan-1970 00:01	19968
All NT passwords.txt	01-Jan-1970 00:01	28941
All NT passwords.xls.pdf	01-Jan-1970 00:01	48415
Arriba User ID and Password memo.doc.pdf	01-Jan-1970 00:01	92576
Arriba User ID and Password memo.txt	01-Jan-1970 00:01	3545
BPR password list.txt	01-Jan-1970 00:01	36228
BPR password list.xls.pdf	01-Jan-1970 00:01	44054
Badri SPF Passwords(1).xlsx	01-Jan-1970 00:01	107008
Black Marks - SAG Actor List password EPCAN-lis...>	01-Jan-1970 00:01	34816
Change password to access TWS server.doc.pdf	01-Jan-1970 00:01	19967
Change password to access TWS server.txt	01-Jan-1970 00:01	668
Computer Passwords.txt	01-Jan-1970 00:01	403
Computer Passwords.xls.pdf	01-Jan-1970 00:01	18001
Contact List and Passwords.txt	01-Jan-1970 00:01	1683
Contact List and Passwords.xls.pdf	01-Jan-1970 00:01	4442
Credit Review Website, Login and Password Info.txt	01-Jan-1970 00:01	4415
Credit Review Website, Login and Password Info...>	01-Jan-1970 00:01	54018
DAAF Passwords.doc.pdf	01-Jan-1970 00:01	32908
DAAF Passwords.txt	01-Jan-1970 00:01	757
DBA Phone List.txt	01-Jan-1970 00:01	1236
DBA Phone List.xls.pdf	01-Jan-1970 00:01	29541
Database Passwords.txt	01-Jan-1970 00:01	150091
Database Passwords.xls.pdf	01-Jan-1970 00:01	292796
Digital User Name and passwords.txt	01-Jan-1970 00:01	3876
Digital User Name and passwords.xls.pdf	01-Jan-1970 00:01	47532
EA userids and passwords.xlsx	01-Jan-1970 00:01	12872
Excel Passwords.xlsx	01-Jan-1970 00:01	7794
FTP LOGIN PASSWORD SHEET.txt	01-Jan-1970 00:01	11828
FTP LOGIN PASSWORD SHEET.xls.pdf	01-Jan-1970 00:01	81719

Figure 5 – Directory listing showing multiple files with password-suggestive filenames.

IMPACT ASSESSMENT

Observed: A publicly accessible directory lists multiple files whose filenames suggest they may contain stored credentials.

Demonstrated: The file listing and filenames were observed. The file contents were not opened, and no credentials were extracted, tested, or used to authenticate to any system.

Potential: If the underlying files do contain valid, still-active credentials, an unauthorized party could potentially obtain authentication information for user accounts, servers, databases, FTP services, or administrative systems. This has not been verified and should be treated as a filename-based observation rather than a confirmed credential leak until the content is reviewed by an authorized party.

RECOMMENDATION

- Remove sensitive files from the public directory where they are not intended to be published.
- Disable directory listing and restrict access to sensitive documents.
- Rotate any credentials that are confirmed to be exposed and still valid.
- Review access logs for signs of prior unauthorized access.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. File and directory names were observed via the browser. No files were opened, downloaded, or used to attempt authentication.

Responsible Disclosure Message :

Subject: **Potential Exposure of Password-Related Files**

Hello Security Team,

During security reconnaissance, I discovered a publicly accessible directory containing multiple password-related files. Some filenames suggest that the files may contain sensitive credentials.

I have not attempted to use any potentially exposed credentials.

I recommend removing the files, restricting directory access, rotating potentially exposed credentials, and reviewing access logs.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

F-06 — Publicly Indexed Administrative Login Page

Category	Informational Observation
Severity	Informational
CWE	CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)
OWASP Category	Not applicable - attack-surface visibility, not a vulnerability in itself
Target	Undisclosed host
Path / URL	/index.php/admin
Discovery Method	Passive reconnaissance (search-engine indexing)
Dork Used	inurl:"admin"

DESCRIPTION

An administrative login page was discovered through a search-engine query. The page displays an Admin Email and Password login form and reveals the /index.php/admin URL path. The presence of a reachable login form is not, by itself, a vulnerability; the observation is that the page is indexed and externally discoverable.

EVIDENCE:

 District Headquarter Hospital
https://dhhodisha.in › index.php › admin
Admin Login
Admin Login. [Read more](#)

 GitHub
https://github.com › google-dorks › blob › main › googl...
google-dorks-for-login.txt
site:target.com inurl:admin | inurl:administrator | inurl:adm | inurl:login | inurl:l0gin | inurl:wp-login
inurl:wp-login inurl:login inurl:user-login ... [Read more](#)

 American Pistachio Growers
http://carboncalculator.americanpistachios.org › admin
Admin Login
Admin Login. Username. Password. [Read more](#)

 Nagar Nigam Mathura Vrindavan
https://nnmsonline.in › admin
Admin Login
Blumoon Admin Dashboard Logo. Admin Login. User Id : Password : [Read more](#)

 Exploit-DB
https://www.exploit-db.com › ghdb
inurl "/admin/index.php" username=admin&password= ...
inurl "/admin/index.php" username=admin&password=password. GHDB-ID: 7149. Author: Sahil Gupta.
Published: 2021-09-02 ... [Read more](#)

 Scribd
https://www.scribd.com › document › the-ultimate-dorks
Admin Login URL Patterns | PDF
7 Aug 2025 — The document contains a comprehensive list of URL patterns commonly associated with admin login pages across various web platforms. [Read more](#)

 GANADIPATHY TULSI'S JAIN ENGINEERING COLLEGE
http://gtec.ac.in › admin
Admin Login Form
Username: Password: Login. [Read more](#)

IMPACT ASSESSMENT

Observed: The administrative login form is publicly reachable and indexed by search engines, revealing the organization's admin URL structure.

Demonstrated: No credentials were entered, guessed, or brute-forced against the login form. No authentication bypass was attempted.

Potential: Knowledge of an admin portal's existence and URL structure can help an attacker map the organization's administrative attack surface and plan more targeted reconnaissance, credential attacks, or social engineering. This is a reconnaissance aid rather than a demonstrated weakness in the login mechanism itself.

RECOMMENDATION

- Review whether the admin portal needs to be publicly reachable at all.
- Prevent unnecessary search-engine indexing of administrative pages (robots.txt / noindex).
- Remove unused or legacy admin interfaces.
- Require MFA and strong authentication for administrative accounts.
- Monitor administrative login activity for anomalies.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. No credentials, login attempts, brute force, bypass, scanning, or exploitation were performed.

Responsible Disclosure Message :

Subject: Publicly Indexed Administrative Login Page

Hello Security Team,

During search-engine reconnaissance, I found that an administrative login page at `/index.php/admin` is indexed by search engines and publicly reachable, revealing the organization's admin URL structure.

No attempt was made to access, modify, or exploit the exposed information beyond confirming the finding.

I recommend reviewing whether this admin portal needs to remain publicly accessible, preventing unnecessary indexing, and enforcing MFA for administrative accounts.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

F-07 — Information Disclosure via robots.txt

Category	Informational Observation
Severity	Low
CWE	CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)
OWASP Category	A05:2021 - Security Misconfiguration
Target	who.int
Path / URL	/robots.txt
Discovery Method	Passive reconnaissance (search-engine indexing)
Dork Used	inurl:robots filetype:txt

DESCRIPTION

The robots.txt file at the target host publicly lists crawler directives and references to legacy or historical paths. robots.txt is intended to be publicly readable by design, so this finding concerns the level of detail disclosed rather than the file's mere existence.

EVIDENCE

```

### Version Information #
#####
### Version: V3.2018.05.828
### Updated: Tue May  8 11:37:04 SAST 2018
### Bad Bot Count: 527
#####
### Version Information ##

User-agent: 360Spider
Disallow:/
User-agent: 404checker
Disallow:/
User-agent: 404enemy
Disallow:/
User-agent: 80legs
Disallow:/
User-agent: Abonti
Disallow:/
User-agent: Aboundex
Disallow:/
User-agent: Acunetix
Disallow:/
User-agent: ADMantX
Disallow:/
User-agent: AfD-Verbotsverfahren
Disallow:/
User-agent: AhrefsBot
Disallow:/
User-agent: AIBOT
Disallow:/
User-agent: AiHitBot
Disallow:/
User-agent: Aipbot
Disallow:/
User-agent: Alexibot
Disallow:/
User-agent: Alligator
Disallow:/
User-agent: AllSubmitter
Disallow:/
User-agent: AlphaBot
Disallow:/
User-agent: Anarchie
Disallow:/
User-agent: Apexoo
Disallow:/
User-agent: ASPSeek
Disallow:/
User-agent: Asterias
Disallow:/
User-agent: Attach
Disallow:/
User-agent: autoemailspider
Disallow:/
User-agent: BackDoorBot
Disallow:/
User-agent: Backlink-Ceck
Disallow:/
User-agent: backlink-check
Disallow:/
User-agent: BacklinkCrawler
Disallow:/

```

Figure 7 – Publicly viewable robots.txt content listing crawl directives and paths.

IMPACT ASSESSMENT

Observed: The robots.txt file is reachable, as expected for this file type, and lists a number of crawler rules and legacy path references.

Demonstrated: The file was viewed directly. No paths listed in the file were accessed or probed further.

Potential: An attacker could use the listed paths to identify legacy or lesser-known areas of the application worth investigating further, aiding reconnaissance and target profiling.

RECOMMENDATION

- Review the file for unnecessary detail, particularly references to legacy or sensitive paths.
- Remove disallow entries that inadvertently advertise the existence of sensitive paths, since disallowed paths remain visible to anyone who reads the file.
- Rely on authentication and access control, not robots.txt, to protect any path that must stay private.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. No credentials, login attempts, brute force, bypass, scanning, or exploitation were performed.

Responsible Disclosure Message :

Subject: **Information Disclosure via robots.txt**

Hello Security Team,

During passive reconnaissance, I found that the robots.txt file at <https://www.who.int/robots.txt> publicly reveals crawler configuration and legacy path information that could assist an attacker in profiling the site.

No attempt was made to access, modify, or exploit the exposed information beyond confirming the finding.

I recommend reviewing the file for unnecessary detail and removing references to any sensitive or legacy paths.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

F-08 — TxDOT Public Plans Directory Listing

Category	Informational Observation (Likely Intentional)
Severity	Informational
CWE	CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)
OWASP Category	Not applicable - appears to be an intentional public document system
Target	ftp.txdot.gov
Path / URL	/plans/
Discovery Method	Passive reconnaissance (search-engine indexing)
Dork Used	"index of /" site:gov

DESCRIPTION

A publicly reachable folder was found at [ftp.txdot.gov/plans/](ftp://ftp.txdot.gov/plans/), titled "Index of /plans", listing several subfolders without requiring a login: Airport, Facilities, Local-Let-Maintenance, State-Let-Construction, State-Let-Maintenance, along with two supporting folders named lib and utils, each with a last-modified date. This closely resembles an intentional public document system that TxDOT uses to share project plans, rather than a hacked or misconfigured private system.

EVIDENCE

Index of /plans

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory		-	
 Airport/	2026-06-04 14:51	-	
 Facilities/	2025-11-17 10:09	-	
 Local-Let-Maintenance/	2025-12-11 08:50	-	
 State-Let-Construction/	2025-12-05 07:07	-	
 State-Let-Maintenance/	2025-12-15 07:47	-	
 lib/	2025-03-10 20:30	-	
 utils/	2025-03-10 20:30	-	

Figure 8 – Directory listing of [ftp.txdot.gov/plans/](ftp://ftp.txdot.gov/plans/) showing publicly shared folders.

IMPACT ASSESSMENT

Observed: The folder structure and naming conventions used by TxDOT for project-plan distribution are visible without authentication.

Demonstrated: The directory listing was viewed only; individual files within the folders were not opened or reviewed.

Potential: If some of the listed folders were not intended to be public, or if they contain files beyond the intended project-plan documents, the exposure could aid reconnaissance of the organization's file layout and naming conventions. Because the structure strongly resembles a deliberate public-records system, this is presented as an observation for confirmation rather than a confirmed vulnerability.

RECOMMENDATION

- Confirm that all listed folders are intended to be public; if so, no remediation is required beyond periodic review.
- Review folder contents for any unintended files (configuration data, credentials, internal-only documents, PII) and remove or relocate them.
- Where some folders are intentionally public and others are not, separate them logically and apply authentication to the non-public ones.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. No attempt was made to bypass authentication, access restricted systems, modify content, or obtain information beyond what was already publicly listed.

Responsible Disclosure Message :

Subject: **Directory Listing Exposure on ftp.txdot.gov**

Hello Security Team,

During passive reconnaissance, I found that ftp.txdot.gov/plans/ has directory listing enabled, exposing the organization's internal folder structure and naming conventions.

No attempt was made to access, modify, or exploit the exposed information beyond confirming the finding.

I recommend disabling directory indexing where it is not intentionally required, and reviewing folder contents for anything not meant to be public.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

F-09 — Publicly Indexed Subdomains and Login Portals

Category	Informational Observation
Severity	Low
CWE	CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)
OWASP Category	A05:2021 - Security Misconfiguration
Target	*.wipro.com
Path / URL	Multiple indexed subdomains
Discovery Method	Passive reconnaissance (search-engine indexing)
Dork Used	site:*.wipro.com intitle:login

DESCRIPTION

Passive reconnaissance identified multiple publicly accessible subdomains and login portals indexed by search engines. These pages may reveal internal applications, administrative interfaces, legacy/test environments, third-party integrations, authentication mechanisms, and underlying software platforms. Even where the portals do not permit unauthorized access, their public discoverability provides reconnaissance value to an external party.

EVIDENCE

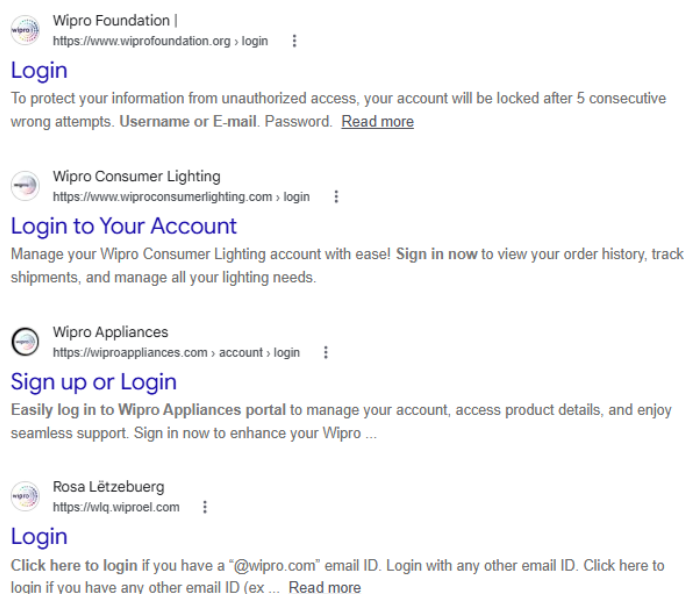


Figure 9 – Search-engine results listing multiple indexed *.wipro.com login portals.

IMPACT ASSESSMENT

Observed: A number of subdomains and login-portal pages under the target's domain are indexed and publicly discoverable through search engines.

Demonstrated: No login attempts, credential testing, or access attempts were made against any of the identified portals.

Potential: Login-page branding, URL structures, and software identifiers can help an attacker identify technologies in use, craft convincing phishing pages, or select credential-stuffing/password-spraying targets. Forgotten legacy or third-party portals may also remain reachable longer than intended.

RECOMMENDATION

- Review identified login portals and confirm each is authorized to be publicly accessible.
- Apply rate limiting, account lockout/step-up controls, and monitoring against credential-stuffing and password-spraying activity.
- Apply noindex controls to portals that must remain reachable but do not need to appear in search results.
- Periodically re-run passive reconnaissance to validate that remediation is holding.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. No credentials, login attempts, brute force, bypass, scanning, or exploitation were performed.

Responsible Disclosure Message :

Subject: **Publicly Indexed Subdomains and Login Portals**

Hello Security Team,

During passive reconnaissance, I found that multiple subdomains and login portals under wipro.com are indexed by search engines, exposing authentication endpoints and branding details that could aid reconnaissance or phishing.

No attempt was made to access, modify, or exploit the exposed information beyond confirming the finding.

I recommend reviewing which portals need to remain public, applying noindex controls where appropriate, and monitoring for credential-stuffing activity.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

Observed: The financial document is indexed by search engines and directly downloadable without authentication.

Demonstrated: The document's existence and public reachability were confirmed. Its contents were reviewed only to the extent needed to identify what type of document it is.

Potential: If this document is intended to be publicly available (for example, as part of a statutory filing), this finding may reflect expected behavior and not represent a security vulnerability. If it was not intended for unrestricted public access, its indexing and direct accessibility could constitute unintended information disclosure and make the information easier for unauthenticated parties to discover.

RECOMMENDATION

- Verify whether this document is intentionally published as a public corporate filing.
- If not intended to be public, remove it from public access or place it behind appropriate access controls.
- Review other documents in the same directory for similar unintended exposure.
- Request removal of unintended copies from search-engine indexes where appropriate.

TESTING SCOPE / VALIDATION

Passive reconnaissance only. No attempt was made to access, modify, or exploit the document beyond confirming the finding.

Responsible Disclosure Message :

Subject: Publicly Accessible Corporate Financial Document

Hello Security Team,

During passive reconnaissance, I found a PDF titled “WIPRO DO BRASIL SERVIÇOS LTDA. – Balance Sheets” that is indexed by search engines and directly accessible without authentication, containing corporate financial statement details.

No attempt was made to access, modify, or exploit the exposed information beyond confirming the finding.

I recommend confirming whether this document is intended to be public and, if not, removing it from public access and requesting removal from search-engine indexes.

I can provide the affected URL and screenshot as evidence through a secure channel.

Regards,
Amaresh Javali

8. Conclusion

This assessment identified 10 observations through passive, search-engine-based reconnaissance, of which 5 represent confirmed security vulnerabilities warranting prompt remediation and 5 represent informational or reconnaissance-value observations. The two Critical-severity findings — the unauthenticated camera feed (F-03) and the exposed database credentials (F-04) — pose the most immediate risk and are recommended for same-week remediation.

None of the findings in this report required credential use, exploitation, or any activity beyond viewing already-indexed, publicly reachable content. This is consistent with the assessment's passive scope and means that the actual, real-world impact of several findings — particularly F-05's password-suggestive filenames and F-02's Git repository contents — remains unverified and should be confirmed by an authorized party before further action is taken. Recommendations in Section 7 are prioritized to address the highest-confidence, highest-impact issues first.

Google Dorking Methodology Reference:

Google dorking, also known as Google hacking, is the practice of using advanced search operators to filter search results down to content that is technically public but not easy to find through an ordinary query. A dork does not break into a website, bypass a login, or exploit a vulnerability — it only makes an existing search engine more precise, in the same way any advanced search feature would.

WHY IT IS CALLED A "DORK"

The term became popular in the information-security community after researchers noticed that search engines routinely indexed sensitive files and pages that site owners had left exposed by mistake, such as backups, credentials, and admin panels. "Dork" originally referred to the careless site owner rather than the technique itself, though today the word is used for the search queries as well.

CORE SEARCH OPERATORS

- **site:** — Restricts results to a specific website or domain. Useful for enumerating indexed pages and subdomains belonging to one organization. Example: `site:example.com`
- **filetype:** — Restricts results to a specific file extension, such as PDF, XLSX, or TXT. Commonly used to locate documents, backups, and data exports. Example: `filetype:pdf "confidential"`
- **intitle:** — Restricts results to pages whose HTML title contains the given keyword. Often used to find pages with generic, revealing titles. Example: `intitle:"index of"`
- **inurl:** — Restricts results to pages whose URL contains the given keyword. Useful for locating specific paths such as login or admin panels. Example: `inurl:admin`
- **intext:** — Restricts results to pages containing the given word or phrase in the visible body text, rather than the title or URL. Example: `intext:"password"`
- **" "** (exact phrase) — Quotation marks force an exact-phrase match instead of a loose keyword match, sharply narrowing results. Example: `"internal use only"`
- **-** (minus) — Excludes a term from the results, filtering out irrelevant matches that share a keyword. Example: `site:example.com -inurl:blog`
- ***** (wildcard) — Acts as a placeholder for one or more unknown words, useful when part of a phrase or path is unknown. Example: `index of * password`

COMBINING OPERATORS FOR PRECISION

Operators can be combined in a single query to narrow results far more than any one operator alone. For example, `site:example.com filetype:env intext:"DB_PASSWORD"` looks specifically for exposed environment files on one domain that contain a database-password variable, rather than searching the entire web.

COMMON DORK CATEGORIES

- **Open directories** — unindexed folders left browsable, often revealing source code, backups, or media. Example: `intitle:"index of" ".git"`
- **Login and admin portals** — authentication pages that are indexed and reachable, useful for mapping attack surface. Example: `inurl:admin intitle:login`
- **Configuration and secrets files** — environment files, database dumps, or credential files accidentally published. Example: `filetype:env "DB_PASSWORD"`

- Error messages and stack traces — pages that leak framework versions, file paths, or internal logic through unhandled errors. Example: `intext:"Warning: mysql_connect()"`
- Exposed devices — cameras, printers, and IoT admin panels deployed with default, internet-facing configurations. Example: `inurl:/view/view.shtml`
- Cloud storage buckets — misconfigured object-storage buckets that allow public listing or download. Example: `site:s3.amazonaws.com "index of"`

THE GOOGLE HACKING DATABASE (GHDB)

The Google Hacking Database, maintained on Exploit-DB, is a public, categorized collection of dorks submitted by researchers over the years. It is a useful reference for understanding which query patterns reliably surface which kinds of exposure, and it is commonly cited in security-awareness training and penetration-testing methodology.

HOW ATTACKERS USE GOOGLE TO GATHER INTELLIGENCE

Search engines index anything left publicly reachable, and Google dorks simply make that indexed content easy to filter and find. The technique itself does not break into anything — it only searches more precisely than a normal query.

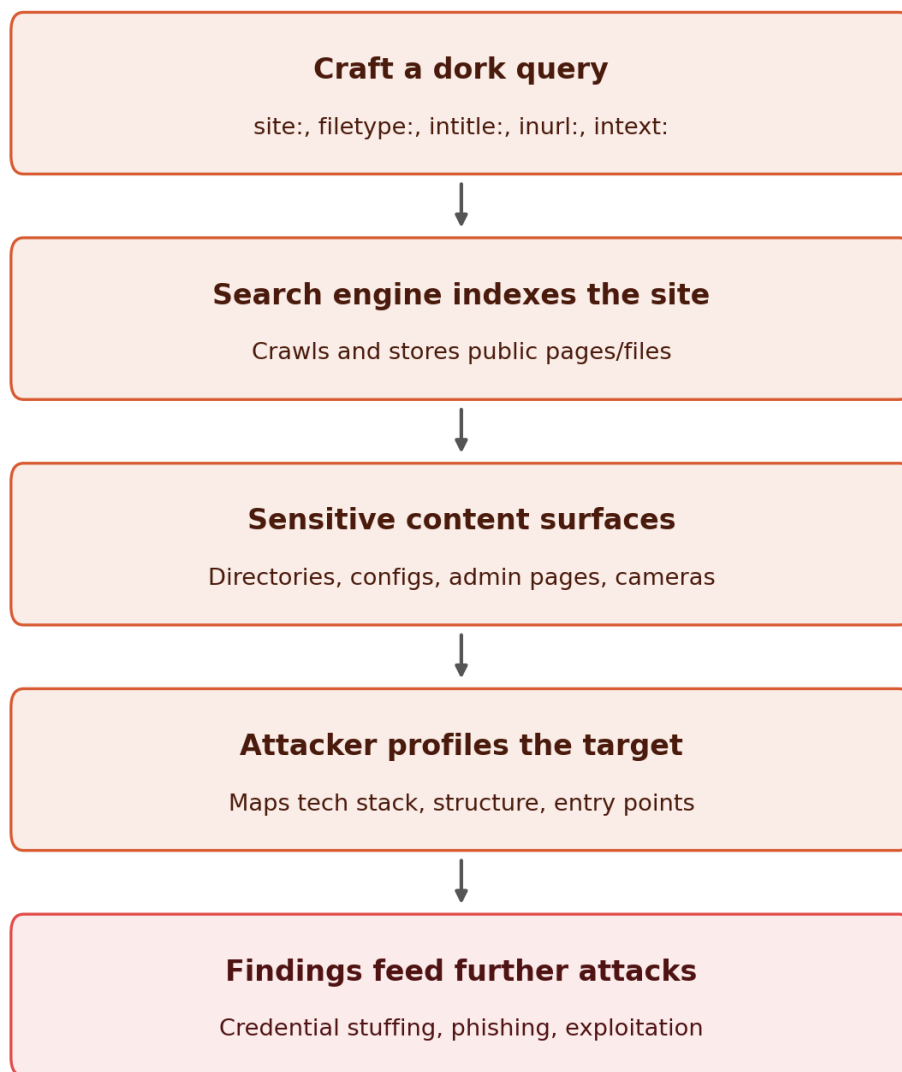


Figure 11 – Typical flow from crafting a dork query to attacker follow-on activity.

- Craft a precise dork query using operators such as site:, filetype:, intitle:, inurl:, and intext:.
- The search engine crawls and indexes public pages and files that match the query.
- Sensitive content surfaces: open directories, admin panels, configuration files, exposed cameras, or leaked credentials.
- The attacker reviews the findings to profile the target's technology, structure, and possible entry points.
- The findings feed further activity, such as credential attacks, phishing, or targeted exploitation.

HOW WEB ADMINS CAN REDUCE EXPOSURE

Most of the findings in this report are reachable purely because content was indexed and left open. The same categories of dorks used to find exposure can be used by an organization to find and close its own gaps.

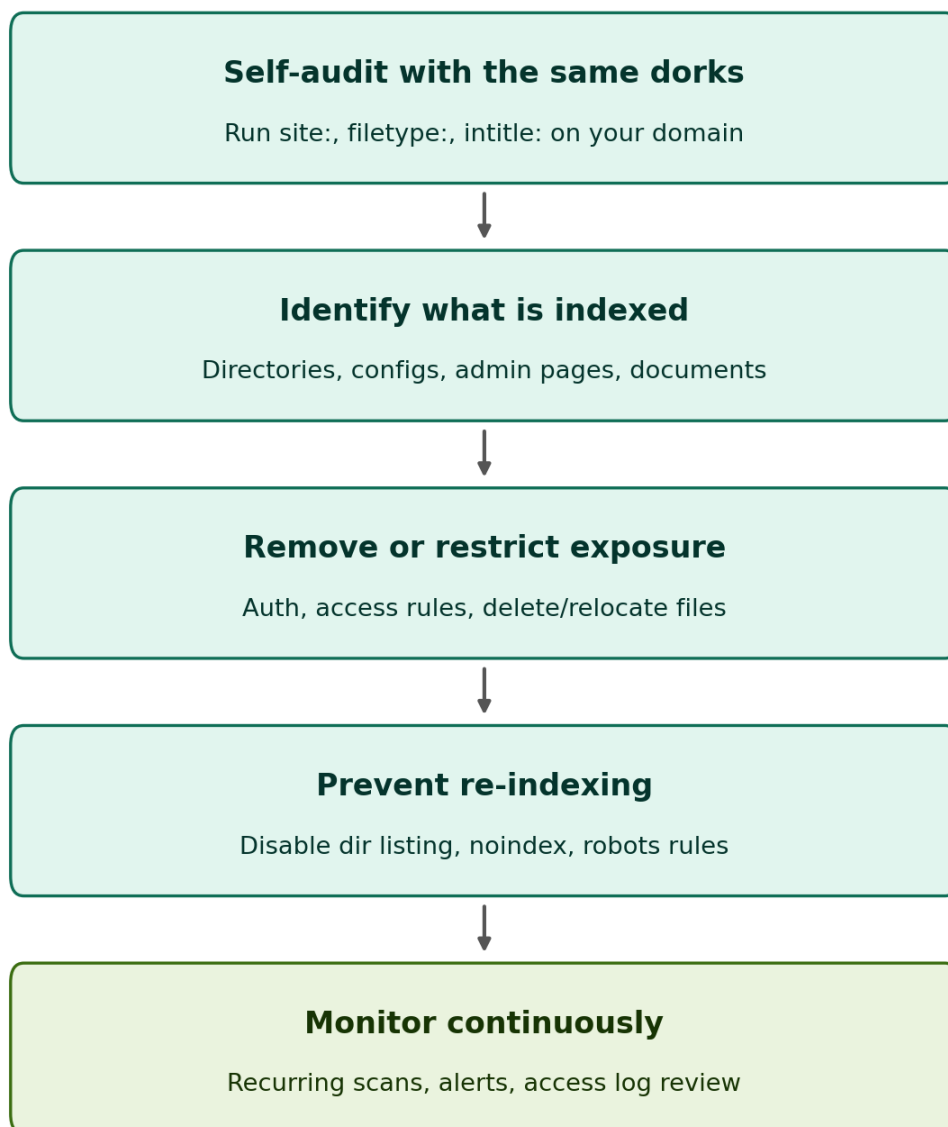


Figure 12 – Recommended self-audit cycle for reducing an organization's own search-engine exposure.

- Run the same dorks against your own domain on a regular basis to see what is publicly indexed.
- Identify what has been indexed: open directories, configuration files, admin pages, internal documents.
- Remove or restrict exposed content using authentication, access rules, or by deleting/relocating files.
- Prevent re-indexing by disabling directory listing, adding noindex tags, and tightening robots.txt.
- Monitor continuously with recurring scans, search-engine alerts, and access-log review.

DETECTING AND LIMITING YOUR OWN EXPOSURE

- Use Google Search Console to review what Google has indexed for your domain and to submit removal requests for content that should not be public.
- Add an X-Robots-Tag: noindex header or a robots.txt disallow rule to paths that must stay reachable internally but should never appear in search results.
- Removing a file does not remove it from a search engine's cache or from third-party archives (such as the Wayback Machine) — request removal from those separately if the content was sensitive.
- Periodically re-run your own dorks; new files, backups, or misconfigurations get indexed continuously, so a one-time cleanup is not sufficient.

RESPONSIBLE USE REMINDER

Everything a dork surfaces was already published and indexed by the search engine; the query itself changes nothing about the target. Using these operators to look up general information or to audit your own organization's exposure is normal security practice. Using them to locate and access another party's non-public data, credentials, or systems is unauthorized activity and may be illegal, regardless of how the information was found.

